

accounts



opening a file?

```
open("/u/bradjc/private.txt", O_RDONLY)
```

say, private file on portal

on Linux: makes system call

kernel needs to decide if this should work or not



system call = ???

(more detail in later lecture)

way for programs to make request to OS:

program sets registers to indicate requested operation

example: “open foo.txt”

“system call number” (open) and “arguments” (foo.txt)

program runs special instruction (x86-64: `syscall`)

hardware switches into privileged mode + runs OS function

OS function decodes requested operation from registers

OS function decides if operation is allowed to proceed



deciding whether syscall should proceed?

```
open("/u/bradjc/private.txt", O_RDONLY)
```

argument: needs extra metadata

what would be wrong with using...

system call arguments?

where the code calling open came from?



user IDs

most common way OSes identify “who” process belongs to:

process = instance of running program (w/ own registers+memory)
(we'll be more specific about processes later)

(unspecified for now) procedure sets user IDs

every process has a user ID

user ID used to decide what process is authorized to do



POSIX user IDs

```
uid_t geteuid(); // get current process's "effective" user ID
```

process's user identified with unique number

core part of OS only knows number (not name!)

- core, always loaded part of OS = “kernel”

- the part of the OS with extra privs with hardware

- the part of the OS that enforces program restrictions

effective user ID is used for all permission checks

also some other user IDs

standard programs/library maintain number to name mapping

- /etc/passwd on typical single-user systems

- network database on department machines



POSIX groups

```
gid_t getegid(void);  
    // process's "effective" group ID  
  
int getgroups(int size, gid_t list[]);  
    // process's extra group IDs
```

POSIX also has group IDs

like user IDs: kernel (= core part of OS) only knows numbers

standard library+databases for mapping to names

also process has some other group IDs — we'll talk later



id command

```
: /u/bjc8c ; id  
uid=858545(bjc8c) gid=90002(csfaculty)  
groups=90002(csfaculty),150015(slurm-cs-brad-campbell)
```

id command displays uid, gid, group list

names looked up in database

kernel doesn't know about this database
code in the C standard library



groups that don't correspond to users

example: video group for access to monitor

put process in video group when logged in directly

don't do it when SSH'd in

... but: user can keep program running with video group
in the background after logout?

id and group id establish identity

who is running a process or accessing a resource

but also need permissions

what users can access



POSIX file permissions

POSIX files support a per-file access control list

interface is very narrow/limited

binary permissions are read, write, and execute

set for three categories: user, group, other

one user ID + read/write/execute bits for user

“owner” — also can change permissions

one group ID + read/write/execute bits for group

default setting — read/write/execute

on directories, ‘execute’ means ‘search’ instead



permissions encoding

permissions encoded as 9-bit number, can write as octal: XYZ

octal divides into three 3-bit parts:

user permissions (X), group permissions (Y), other permission (Z)

each 3-bit part has a bit for 'read' (4), 'write' (2), 'execute' (1)

binary number: 0bRWE

700 — user read+write+execute; group none; other none

451 — user read; group read+execute; other execute



chmod — exact permissions

```
chmod 700 file  
chmod u=rwx,og= file
```

user read write execute; group/others no access

```
chmod 451 file  
chmod u=r,g=rx,o=x file
```

user read; group read/execute; others execute



chmod — adjusting permissions

```
chmod u+rx foo
```

add user read and execute permissions
leave other settings unchanged

```
chmod o-rwx,u=rx foo
```

remove other read/write/execute permissions
set user permissions to read/execute
leave group settings unchanged



POSIX/NTFS ACLs

more flexible access control lists

list of (user or group, read or write or execute or ...)

supported by NTFS (Windows)

a version standardized by POSIX, but usually not supported

POSIX ACL syntax

```
# group students have read+execute permissions
group:students:r-x
# group faculty has read/write/execute permissions
group:faculty:rwx
# user mst3k has read/write/execute permissions
user:mst3k:rwx
# user tj1a has no permissions
user:tj1a:---

# POSIX acl rule:
# user take precedence over group entries
```



POSIX ACLs on command line

```
getfacl file
```

```
setfacl -m 'user:tj1a:---' file
```

add line to ACL

```
setfacl -x 'user:tj1a' file
```

REMOVE line from acl

```
setfacl -M acl.txt file
```

add to acl, but read what to add from a file

```
setfacl -X acl.txt file
```

remove from acl, but read what to remove from a file



ACLs establish permissions

identity: who is running a process or accessing a resource

permissions: who has what access to a file or resource

now need enforcement

ensure that permissions are implemented correctly



authorization checking on Unix

request made to core part of OS = system call

handler for system calls checks permissions

no relying on libraries, etc. to do checks

files (open, rename, ...) — file/directory permissions include UID or GID

processes (kill, ...) — process UID = user UID

...



superuser

user ID 0 is special

superuser or root

(non-Unix) or Administrator or SYSTEM or ...

some OS functionality: only work for uid 0

shutdown, mount new file systems, etc.

automatically passes all (or almost all) permission checks



superuser v kernel mode

processor has two modes

- kernel mode (what core part of OS uses)

- user mode (every thing else)

programs running as *superuser still in user mode*

- just change in how OS acts when program asks for things

superuser : OS :: kernel mode : hardware

certain hardware instructions/operations only enabled in kernel mode



how does login work?

```
XsomemachineX login: YjoY  
password: ******
```

```
jo@XsomemachineX$ YlsY  
...
```

this is a program which...

checks if the password is correct, and

changes user IDs, and

runs a shell



how does login work?

```
XsomemachineX login: YjoY  
password: ******
```

```
jo@XsomemachineX$ YlsY  
...
```

this is a program which...

checks if the password is correct, and

changes user IDs, and

runs a shell



how does login work?

```
XsomemachineX login: YjoY  
password: ******
```

```
jo@XsomemachineX$ YlsY  
...
```

this is a program which...

checks if the password is correct, and

changes user IDs, and

runs a shell



againframe(loginProgram)



Unix password storage

typical single-user system: /etc/shadow

only readable by root/superuser

department machines: network service

Kerberos / Active Directory:

server takes (encrypted) passwords

server gives tokens: “yes, really this user”

can cryptographically verify tokens come from server



aside: beyond passwords

/bin/login entirely user-space code

only thing special about it: when it's run

could use any criteria to decide, not just passwords

- physical tokens

- biometrics

- ...



againframe(loginProgram)



changing user IDs

```
int setuid(uid_t uid);
```

if superuser: sets effective user ID to arbitrary value
and a “real user ID” and a “saved set-user-ID” (we’ll talk later)

system starts as superuser and login programs run as
superuser

voluntarily restrict own access before running shell, etc.



sudo

```
tj1a@somemachine$ Ysudo restartY  
Password:  ****
```

sudo: run command with superuser permissions

started by non-superuser

recall: inherits non-superuser UID

can't just call `setuid(0)`



set-user-ID sudo

extra metadata bit on executables: set-user-ID

if set: `exec( )` syscall changes effective user ID to owner's ID

“extra” user IDs track what original user was

sudo program: owned by root, marked set-user-ID

sudo's code: if (original user allowed) ...; else print error

marking setuid: `chmod u+s`



uses for setuid programs

mount USB stick

- setuid program controls option to kernel mount syscall
- make sure user can't replace sensitive directories
- make sure user can't mess up filesystems on normal hard disks
- make sure user can't mount new setuid root files

control access to device — printer, monitor, etc.

- setuid program talks to device + decides who can

write to secure log file

- setuid program ensures that log is append-only for normal users

bind to a particular port number $\backslash < 1024$

- setuid program creates socket, then becomes not root



set-user ID programs are very hard to write

what if stdin, stdout, stderr start closed?

what if the PATH env. var. set to directory of malicious programs?

what if `argc == 0`?

what if dynamic linker env. vars are set?

what if some bug allows memory corruption?

...



privilege escalation

privilege escalation — vulnerabilities that allow more privileges

code execution/corruption in utilities that run with high privilege

- e.g. buffer overflow, command injection

- login, sudo, system services, ...

- bugs in system call implementations

logic errors in checking delegated operations

